PÓS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 05 -

IAC EM PIPELINES COM TERRAFORM

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
MERCADO, CASES E TENDÊNCIAS 12
O QUE VOCÊ VIU NESTA AULA? 13
REFERÊNCIAS 14

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VEM POR AÍ?

A infraestrutura como código (IaC) é um pilar estratégico que transforma a gestão de ambientes complexos em um processo programável e auditável. Nesta aula, o foco é total em pipelines de IaC. Você irá aprender a usar o Terraform para construir e gerenciar sua infraestrutura de forma automatizada, segura e consistente em ambientes de CI/CD.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

HANDS ON

As videoaulas irão guiar você na prática, para que o ciclo de vida de uma pipeline de IaC se torne um processo natural. Você irá aprender a configurar um pipeline para executar os comandos init, plan e apply, que são a espinha dorsal de qualquer automação com Terraform. O objetivo é que, ao interagir com os códigos disponibilizados, você se familiarize com a gestão de ambientes e com a forma como o código de infraestrutura é materializado.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

SAIBA MAIS

O Ciclo de Vida do Terraform em Pipelines

A filosofia do Terraform reside em sua abordagem **declarativa**. Em vez de dar instruções passo a passo para criar recursos, você simplesmente define o estado final desejado da sua infraestrutura em arquivos de configuração usando a HashiCorp Configuration Language (HCL). O Terraform “se vira” para descobrir as ações necessárias e entregar o resultado. É simples assim: você foca no "o que" e a ferramenta se encarrega do "como".

O fluxo de trabalho principal do Terraform (Write, Plan e Apply) é a base para qualquer pipeline de IaC.

1. **Write (.tf):** você escreve o código que define seus recursos e as dependências entre eles.
2. **Plan (terraform plan):** pense no plan como o "dry-run" da sua pipeline. Ele compara sua configuração com o estado atual do ambiente e mostra exatamente o que vai ser criado, alterado ou destruído. Essa etapa de segurança permite auditar as mudanças antes que elas aconteçam, o que é fundamental para a governança em ambientes automatizados.

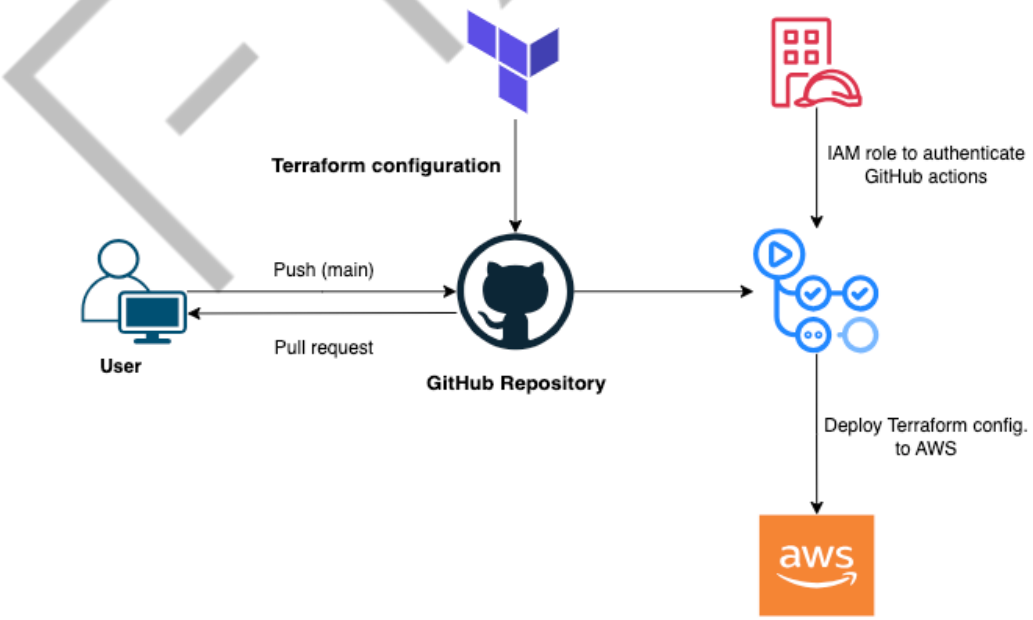


Figura 1 – CI/CD Terraform
Fonte: https://miro.medium.com/v2/resize:fit:670/1*u-Adc9VsZETqT_fJZsWvWQ.png (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

3. **Apply (terraform apply):** a execução do apply é o momento em que a mágica acontece. Ele sincroniza o ambiente real com o código que você escreveu.

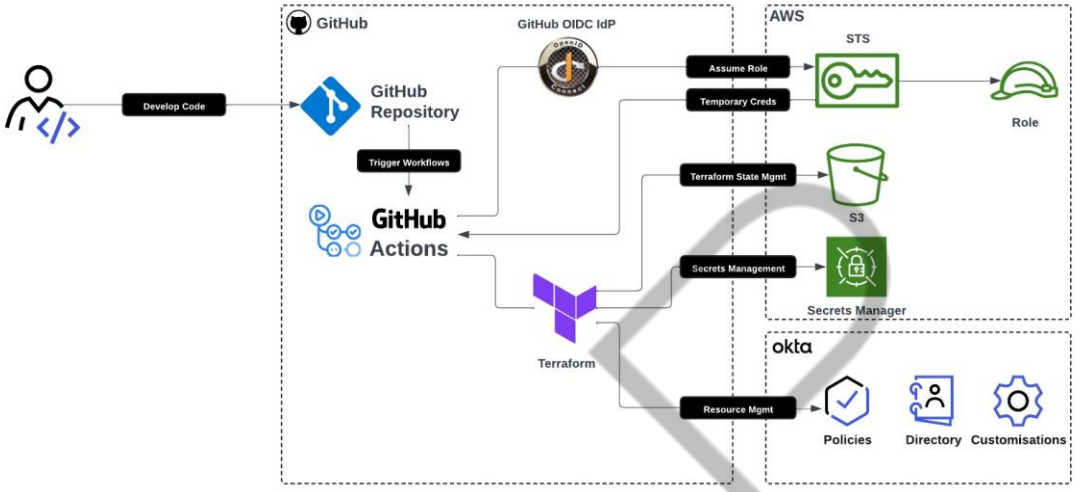


Figura 2 – CI/CD Terraform GitHub Actions
Fonte: https://developer.okta.com/assets-jekyll/blog/terraform-ci-cd/architecture-overview-b47c2b972b6fbb7428f620b5ffe855f07e02c41196b5a1074a766a7571f3c199.jpg (2025)

A automação em pipelines eleva esse fluxo a um novo patamar. O comando terraform plan possui uma opção -out=FILE, que permite salvar o plano de execução em um artefato. Esse arquivo pode ser transportado e executado em um ambiente diferente com terraform apply FILENAME. Essa separação entre a fase de planejamento (que pode ser feita na sua CI para revisões) e a fase de aplicação (que acontece em um ambiente seguro) é o que permite a integração robusta do Terraform em pipelines de CI/CD.

A Filosofia GitOps para Pipelines de IaC

O GitOps é uma abordagem operacional que utiliza o Git como a "única fonte de verdade" para a infraestrutura e a configuração dos ambientes. A ideia central é que o estado desejado do ambiente seja explicitamente definido em arquivos versionados em um repositório Git. Essa filosofia é uma evolução da entrega contínua (CD), mas com uma distinção crucial: ela é declarativa e focada na automação via "pull".

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

No modelo tradicional de CI/CD, conhecido como "push", a pipeline é responsável por "empurrar" as mudanças para o ambiente de destino. Isso exige que a pipeline tenha credenciais de alto privilégio, o que pode representar um risco de segurança. Se a pipeline for comprometida, o ambiente de produção fica em risco.

Em contraste, o modelo "pull" do GitOps elimina essa vulnerabilidade. Nele, a pipeline de CI tem permissão apenas para fazer push para o repositório Git. O processo de implantação é orquestrado por um "agente" (ou operator), que reside dentro do ambiente de destino. Este agente monitora continuamente o repositório Git em busca de novas alterações. Ao detectar uma mudança, ele automaticamente "puxa" a nova configuração e a aplica, garantindo que o estado real corresponda ao estado desejado no Git.

Essa abordagem move as credenciais sensíveis para dentro do ambiente, reduzindo o vetor de ataque. Como cada mudança é um commit no Git, o processo de implantação se torna inerentemente auditável e rastreável, com um histórico completo de todas as alterações feitas no sistema. O GitOps aprimora a automação e a segurança ao centralizar o controle e a rastreabilidade no sistema de controle de versão.

O Coração do Terraform: O Gerenciamento de Estado

Vamos conversar sobre um tema crítico: o **estado** é o componente mais importante do Terraform. O arquivo de estado (terraform.tfstate) é a "fonte única de verdade" que mapeia seu código para os recursos reais na nuvem. Se você o gerencia localmente ele se torna um gargalo e é muito fácil para alguém do seu time corromper esse arquivo ao tentar aplicar mudanças ao mesmo tempo.

Para a colaboração e a segurança, a melhor prática é sempre usar um **estado remoto**. Ao adotar um back-end remoto (como um bucket S3 ou uma solução dedicada como o Terraform Cloud), você centraliza o estado e ganha funcionalidades essenciais para um fluxo de trabalho em equipe.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

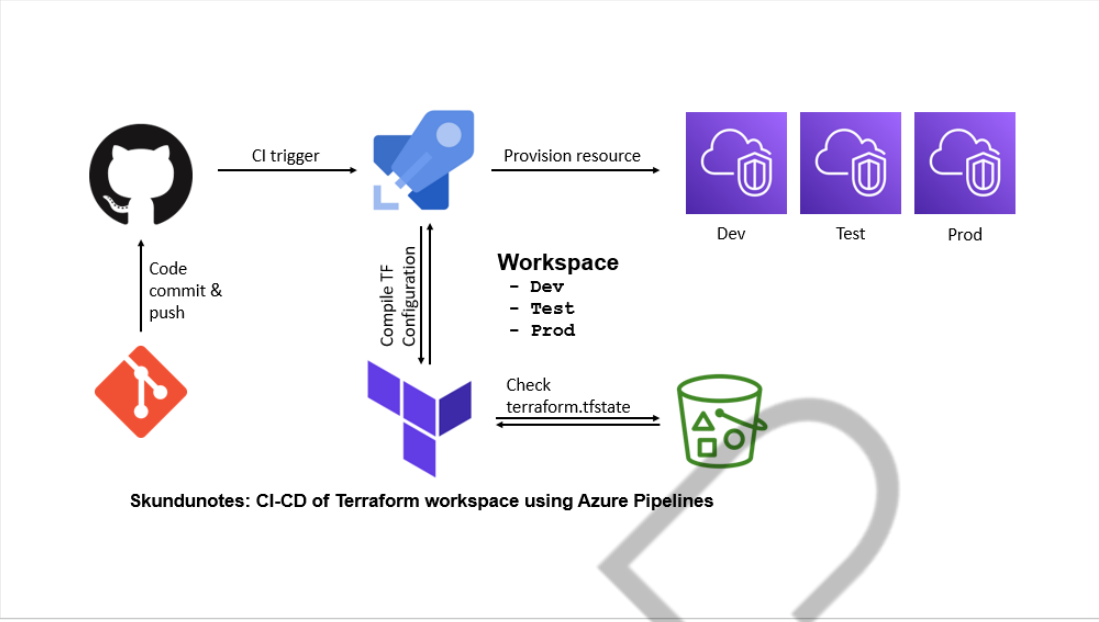


Figura 3 – Terraform State
Fonte: https://skundunotes.com/wp-content/uploads/2021/07/44.-image-1.png (2025)

A tabela a seguir resume as melhores práticas para o gerenciamento de estado remoto, que você deve aplicar em qualquer projeto profissional:

Prática	**Por que fazer isso?**
Usar Estado Remoto	Evita inconsistências e gargalos de colaboração, garantindo uma fonte única de verdade.
Bloqueio de Estado	Previne a corrupção do estado quando várias pessoas tentam aplicar mudanças ao mesmo tempo.
Versionamento e Backups	Permite voltar no tempo, restaurando o estado anterior em caso de erro.
"Plan Locally, Apply Remotely"	Reduz o risco de erros, pois a mudança é revisada antes de ser aplicada no ambiente controlado.
Manter o Estado Pequeno	Aumenta a velocidade e facilita a depuração, evitando arquivos de estado gigantes.

Tabela 1 – Gerenciamento de Estado Terraform
Fonte: elaborado pelo autor (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

Modularidade e Reutilização: Construindo em Escala

Para construir infraestrutura em escala, a modularidade é fundamental. Módulos são contêineres lógicos que agrupam vários recursos que trabalham juntos. Pense neles como "blocos de construção" reutilizáveis: você pode criar um módulo para um cluster Kubernetes e depois usá-lo em diferentes projetos, apenas ajustando as variáveis. Isso organiza o seu código, segue o princípio DRY (Don't Repeat Yourself) e padroniza as implantações na sua empresa.

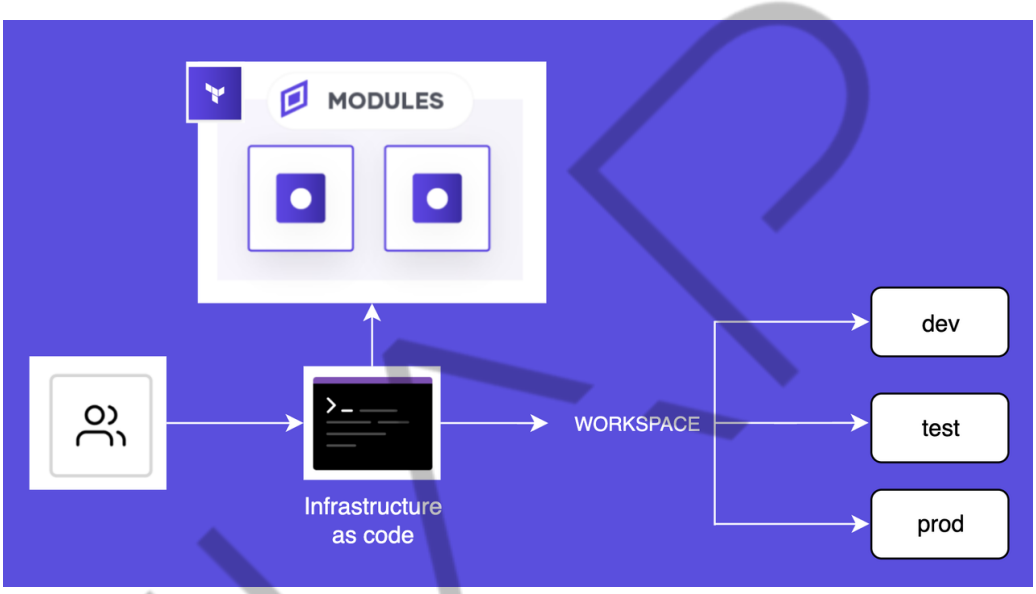


Figura 4 – Terraform Módulos
Fonte: Média (2025)

Um conceito que costuma causar confusão é o de **workspaces**. Workspaces são arquivos de estado nomeados que te permitem gerenciar múltiplas instâncias isoladas da **mesma configuração**. Eles são ótimos para criar ambientes temporários para testes ou depuração, por exemplo.

Segura essa: usar workspaces para gerenciar ambientes de produção, staging e desenvolvimento é um **anti-padrão comum**. Conforme a infraestrutura amadurece, os ambientes raramente permanecem idênticos. As configurações de produção, por exemplo, terão requisitos de segurança e escala diferentes dos ambientes de desenvolvimento. Gerenciar essas diferenças com workspaces e variáveis complexas torna o código difícil de manter e propenso a erros. A melhor prática é usar módulos para o código reutilizável e uma estrutura de diretórios

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

separada para cada ambiente. O uso de workspaces deve ser restrito ao seu propósito original: implantações efêmeras e isoldas para testes.

A distinção clara entre os dois conceitos é fundamental para a tomada de decisões de arquitetura.

Característica	**Módulos**	**Workspaces**
Propósito	Reutilizar e organizar código.	Isolar múltiplas instâncias da mesma configuração.
Reutilização	Baixa, cria blueprints de infra.	Alta, reutiliza a configuração atual.
Gerenciamento de Estado	Não gerenciam o estado.	Criam um arquivo de estado separado.
Casos de Uso Ideal	Componentes reutilizáveis, como uma VPC ou um cluster Kubernetes.	Ambientes temporários para depuração e testes.

Tabela 2 – Modularidade e Reutilização Terraform
Fonte: elaborado pelo autor (2025)

O Gráfico de Dependências: A Inteligência Oculta do Terraform

Por trás dos comandos simples, o Terraform esconde uma inteligência poderosa: o **gráfico de dependências**. O Terraform não executa suas operações na ordem em que você as escreveu. Ele constrói um gráfico que mapeia todas as interconexões e dependências entre seus recursos. Isso garante que um servidor web não seja provisionado antes da rede virtual que o suporta, por exemplo.

Essa inteligência permite que o Terraform execute operações em paralelo, tornando o processo de provisionamento altamente eficiente. Se quiser ver como ele pensa, use o comando terraform graph.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

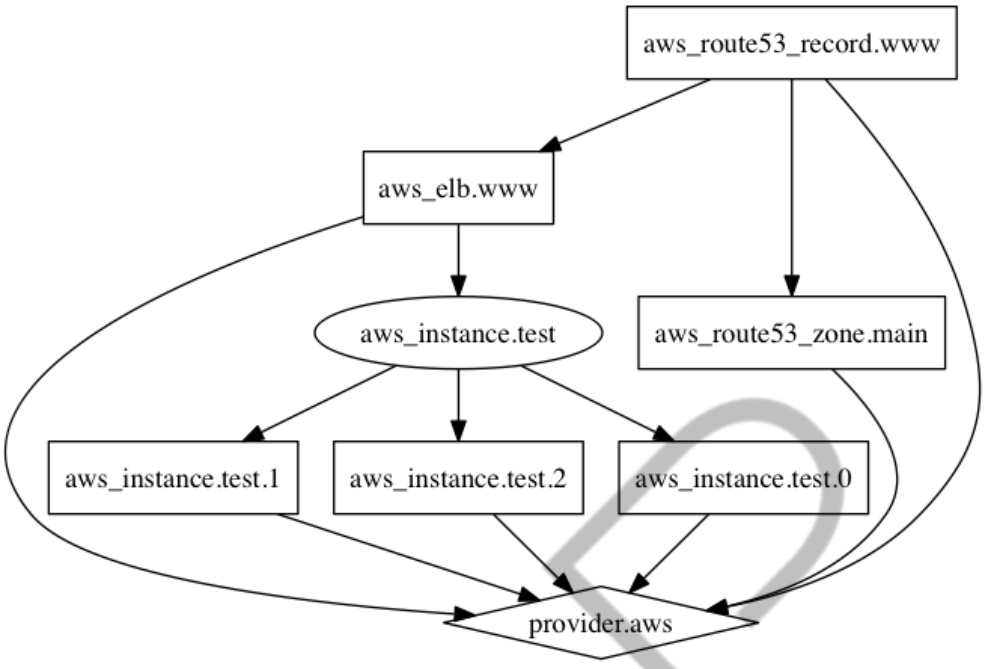


Figura 5 – Terraform Gráfico
Fonte: https://web-unified-docs-hashicorp.vercel.app/api/assets/terraform/latest/img/docs/graph-example.png (2025)

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

MERCADO, CASES E TENDÊNCIAS

Casos de Sucesso e Cenários de Aplicação

A adoção do Terraform vai além da automação básica; ele é a ferramenta de escolha para algumas das arquiteturas de nuvem mais sofisticadas do mundo. Seu uso em implantações **multi-cloud** é um caso de sucesso notável. Ao usar provedores para AWS, Azure e Google Cloud, o Terraform atua como uma camada de abstração que gerencia recursos em diferentes nuvens a partir de uma única base de código, simplificando a orquestração de infraestruturas complexas.

A Evolução do Ecossistema: A Licença BSL e a Ascensão do OpenTofu

O cenário de IaC passou por uma mudança significativa em 2023 quando a HashiCorp anunciou que o Terraform e outras ferramentas populares passariam a operar sob a **Business Source License (BSL)**, em vez da licença open source MPL 2.0. Essa mudança, que não afeta a maioria dos usuários finais que utilizam o Terraform internamente, impôs restrições a empresas que oferecem produtos competitivos aos da HashiCorp. Como resultado, o Terraform deixou de ser considerado uma ferramenta open source pela Open-Source Initiative (OSI).

Em resposta direta a essa decisão, uma coalizão de empresas e a comunidade de código aberto se uniu para criar o **OpenTofu**, um fork do Terraform sob a governança da Linux Foundation. O objetivo do OpenTofu é garantir que uma ferramenta tão fundamental para a infraestrutura moderna permaneça verdadeiramente open source e livre de restrições de licenciamento.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você desmistificou o Terraform e sua filosofia declarativa. Explorou o ciclo de vida fundamental de init, plan e apply e compreendeu como o plan é o componente arquitetural que habilita a automação em escala.

Aprofundamos o conceito de estado, aprendendo por que o gerenciamento remoto é uma prática essencial para a colaboração e a segurança da sua infraestrutura. Discutimos a importância da modularidade, distinguindo módulos de workspaces e identificando os casos de uso ideais para cada um.

Finalmente, a aula o(a) preparou para os desafios e tendências do mercado, analisando o impacto da licença BSL e a ascensão do OpenTofu como uma alternativa open source.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

REFERÊNCIAS

ENV0. **Terraform Remote State Best Practices**. 2025. Disponível em: https://www.env0.com/blog/terraform-remote-state-using-a-remote-backend. Acesso em: 07 out. 2025.

HARNESS. **What is the GitOps workflow**. 2024. Disponível em: https://www.harness.io/blog/what-is-the-gitops-workflow. Acesso em: 07 out. 2025.

HASHICORP. **Terraform CLI**. 2025. Disponível em: https://developer.hashicorp.com/terraform/cli. Acesso em: 07 out. 2025.

HASHICORP. **Terraform Documentation**. 2025. Disponível em: https://developer.hashicorp.com/terraform/docs. Acesso em: 07 out. 2025.

MEDIUM. **Why we are moving away from Terraform in 2025**. 2025. Disponível em: https://medium.com/@engineer-mind/why-we-are-moving-away-from-terraform-2025-76766966bb05. Acesso em: 07 out. 2025.

OPENTOFU. **OpenTofu.** 2025. Disponível em: https://opentofu.org/. Acesso em: 07 out. 2025.

SPACELIFT. **Terraform Workspaces**. 2025. Disponível em: https://spacelift.io/blog/terraform-workspaces. Acesso em: 07 out. 2025.

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com

PALAVRAS-CHAVE

IaC. Terraform. HCL. Infraestrutura como Código. Ciclo de Vida. Estado Remoto. Módulos. Workspaces. OpenTofu. BSL. Automação. DevOps. Arquitetura Cloud.

FIAP

PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com



PDF exclusivo para Vinícius Chiarelo Gomes - rm374954
chiiarelo.dev@gmail.com